

Federated Process Framework in a Virtual Enterprise Using an Object-Oriented Database and Extensible Markup Language

Kyoung-Il Bae
Jung-Hyun Kim
Soon-Young Huh

Korea Advanced Institute of Science and Technology

ABSTRACT

Process information sharing is a beneficial tool through which participating organizations in a virtual enterprise can improve their customer services and business performance. However, autonomy and agility of the organizations have placed limitations in the development of process information sharing, which the previous research has not satisfactorily addressed. This paper proposes a federated process framework and its system architecture that provide a conceptual design for effective implementation of process information sharing supporting the autonomy and agility of the organizations. First, in terms of autonomy, the federated process framework supports a flexible sharing policy to control the amount of shared data so that the framework can be applied to a wide variety of practical situations from loosely-coupled cases to tightly-coupled cases. Second, in terms of agility, the system architecture based on the federated process framework supports the entire life cycle of process information sharing by allowing sufficient adaptability to the changes of business environments. We develop the framework using an object-oriented database and Extensible Markup Language to accommodate all the constructs and their interactions within object-oriented message exchange model.

Keywords: information sharing; federated process framework; object-oriented message exchange model

INTRODUCTION

Recent studies on business-to-business electronic commerce increasingly emphasize effective business collaboration in a virtual enterprise in terms of order fulfillment, marketing, and customer services (D'Amours, 1999; Tuma, 1998). Especially, by streamlining and coordinating collabo-

rating organizations' local business processes in their value chain, business collaboration satisfies customers' demands such as order tracking and just-in-time delivery. The key technique for achieving such goals is process information sharing (Alonso, 1999; Georgakopoulos, 1999). Process information sharing means that collaborating organizations provide visibility of their internal process information to

each other in order to enhance process monitoring capabilities and dynamic resource management coordinated in a virtual enterprise. In the example of an on-line store case, if an on-line store receives detailed delivery process information from a collaborating transportation company, it can effectively carry out and monitor full steps of order fulfillment processes from order capturing through picking and packing, finally to product delivery. Such process information sharing allows the on-line store to provide customers an extended order tracking service to monitor the overall process status for their orders. From the viewpoint of the transportation company, if the on-line store provides order handling process information prior to sending a delivery request, the transportation company can better plan its delivery schedule for car allocation, loading, and routing, and thus reduce the overall delivery time and cost. With such benefits of process information sharing, both the on-line store and transportation company can improve their business performance in a variety of ways including value-added customer service and decision support.

Most of the previous research on process information sharing has focused on demonstrating such benefits (Lee, 1997; D'Amours, 1999; Zhou, 1998) and providing appropriate underlying system architecture or design for process information sharing (Alonso, 1999; Georgakopoulos, 1999; Kuechler, 2001; Mori, 1999; Workflow Management Coalition, 2000). However, research efforts considering the autonomy and agility issues that are the inherent properties of modern organizations are few even though these issues make it difficult to accommodate process information sharing in many real situations.

Motivation and Research Questions

Autonomy means that each participating organization can decide whether to and how much of its local data to share with other organizations (Alonso, 1989; Sheth, 1990). In spite of strategic alliances, most organizations in a virtual enterprise are usually reluctant to expose their core business information on their internal business logic and status to each other (Bolcer, 1999; Georgakopoulos, 1999, Merz, 1999). Such unwillingness often conflicts with the need to share data and therefore these two conflicting factors determine the degree of autonomy for the amount of shared data. When the degree of autonomy is determined, each participant establishes a sharing policy on the amount of shared data. Regarding the autonomy issue, existing studies on process information sharing have addressed little about the mechanisms to establish a sharing policy and how to control the amount of shared process information according to the sharing policy. Specifically, the autonomy problem can be further articulated by the following questions:

- i) How can the amount of shared process information be systematically represented as a sharing policy concerning the determined degree of autonomy between participants?
- ii) How can the system for process information sharing restrict and control the amount of shared process information according to the sharing policy while accommodating seamless process information sharing?

On the other hand, agility means that organizations constantly refine their business strategies and information systems in order to meet environmental changes or to take new opportunities (Goranson, 1999). It is well recognized that agility is one of

the core competencies in the fast-changing modern business environment (Scott-Morton, 1994). In terms of process information sharing, we specifically consider that an agile organization in a virtual enterprise adopts either of the following two methods in response to the changes of business environments: first, the internal business process is amended to meet newly proposed market or organizational constraints or to improve business operations from the advent of new technologies (Casati, 1998; Lee, 1999). Second, the business relationships with other collaborating organizations are modified due to the changes of mutual dependencies or external environments, and corresponding sharing policies are changed accordingly (Bakos, 1998; Chircu, 2000). Since these methods change the schema and amount of shared process information, the system for process information sharing should also be modified and recompiled to reflect the changes incurred in the system procedures for data sharing. The more frequently such a change arises, the more seriously the maintenance cost is to be considered. Such an agility problem can be further delineated by the following questions:

- i) How can the system for process information sharing adapt itself to the change of a participant's internal business process or sharing policy without causing serious maintenance cost?
- ii) In developing the system, which system components are suitable or necessary to accommodate such adaptability?

Research Objective and Adopted Technologies

The main objective of this paper is to propose a federated process framework as a conceptual design to support effective

implementation of process information sharing among collaborating but autonomous organizations while resolving the autonomy and agility problems. In the actual development of process information sharing, implementation issues such as security control and data conflict resolution should be under consideration. However, with a conceptual perspective, this paper focuses on providing answers to the foregoing four research questions related to the autonomy and agility problems.

We employ a federated database system approach (Heimbigner, 1985; Sheth, 1990) and extend it with an Internet-based system architecture. As base technologies, an object-oriented database (OODB) (Kim, 1995) and Extensible Markup Language (XML) (Carter, 2000) are adopted. The OODB allows flexible and natural modeling of business processes and sharing policies while ensuring efficient object persistency with transaction capabilities. Furthermore, classes constituting the object data model entirely or partly generate application programs with object-oriented programming languages. The object data model therefore serves as building blocks both to design a persistent database schema in the OODB and to develop application programs. The XML has been widely adopted as a standard language for communication among distributed software programs (Herring, 2001; Sundaram, 2001; UN/CEFACT, 2001) due to its flexibility and Internet-based architecture. In addition, it provides extensive data manipulation capabilities in the distributed computing environment, such as data integration and granular update from multiple data sources (Böhm, 2000). Because of the resurging popularity and data manipulation capabilities, the XML is adopted in the study as the most appropriate tool for vendor and platform-independent data sharing in the distributed envi-

ronment. To evaluate the effectiveness of the federated process framework, a prototype system has been implemented on a commercial OODB Management System called OBJECTSTORE (Object Design, 2001) with the JAVA programming language (Arnold, 2000).

The paper is organized as follows: the next section introduces the basic concepts of an inter-organizational process model with an order fulfillment example. We then describe the federated process framework and present detailed data designs including an object data model and XML document structure. The fourth section presents an Internet-based system architecture with its prototype system. The final section discusses the results and contributions of this paper and provides future research directions.

INTER-ORGANIZATIONAL PROCESS MODEL

This section briefly introduces basic concepts of an inter-organizational process model and looks at an order fulfillment example to provide the perspective on the research problems of the paper.

Basic Concepts of an Inter-Organizational Process Model

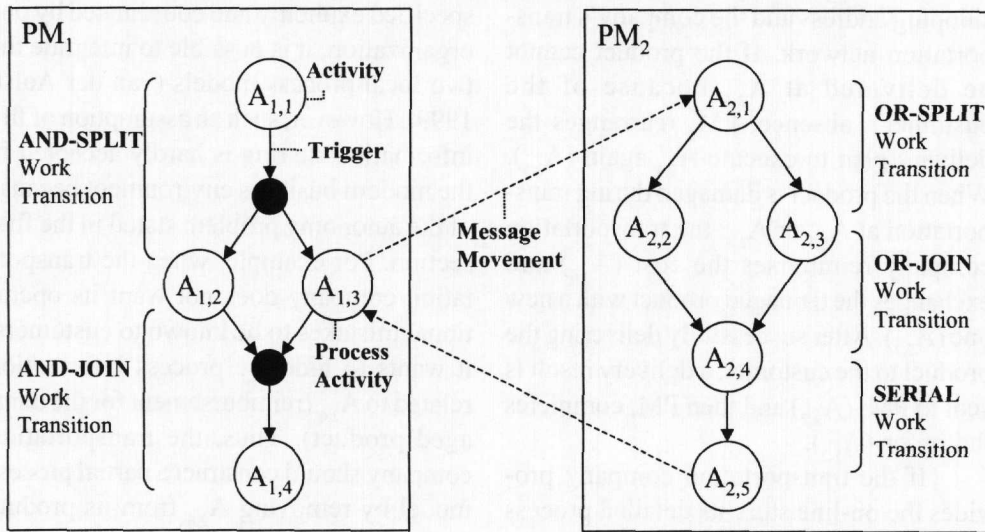
A **process model** is the structure of a business process and all internal work sequences in various conditions. In terms of an object-oriented paradigm, it is a template from which a business process instance is created and executed. The **process status** specifies the status of a particular business process instance. An **inter-organizational process model** is composed of multiple collaborating local process models with their interactions. The

process status on the inter-organizational process model, called a **global process status**, is obtained by merging multiple process statuses from the local process models.

Most previous studies consider an activity and work transition as the most fundamental constructs to represent a process model (Cichocki, 1998; Leymann, 2000; Workflow Management Coalition, 1998). The **activity** is a logical and independent piece of work and the **work transition** represents a flow of a business process among activities. To support process enactment and role assignment, additional constructs such as an execution rule and organization structure need to be defined (Amghar, 2000; Workflow Management Coalition, 1998). However, since the goal of this paper is process information sharing among different organizations that focuses on the monitoring of a global process status, we view that a process model consists of a number of activities and their associating work transitions.

Figure 1 graphically represents the basic concepts of an inter-organizational process model. As notations to depict an inter-organizational process model, PM_i denotes the i -th participating local process model and $A_{i,j}$ denotes the j -th activity of PM_i . In the two local process models, PM_1 and PM_2 , constituting the inter-organizational process, the circle denotes an activity and the solid arrow between circles denotes a **trigger** that represents the order of a work transition. The dotted arrow denotes message movement representing an interaction between different process models. In addition, PM_1 considers PM_2 as a remote sub-process and uses a local activity, $A_{1,3}$, to represent the activities of PM_2 . Such an activity is called a **process activity**. The message from $A_{1,3}$ causes the start of PM_2 ; $A_{1,3}$ is suspended during the ex-

Figure 1: Graphical Representation of an Inter-Organizational Process Model



execution of PM₂; after receiving the returned message from PM₂, A_{1,3} terminates.

Figure 1 also shows five kinds of work transitions including AND-SPLIT, AND-JOIN, OR-SPLIT, OR-JOIN, and SERIAL (Leymann, 2000; Workflow Management Coalition, 1998). The black dot between solid arrows denotes a discriminator to distinguish AND-JOIN from OR-JOIN, and AND-SPLIT from OR-SPLIT. Consider the activity A_{1,1} as an example in Figure 1. When A_{1,1} terminates, both A_{1,2} and A_{1,3} follow in parallel (we denote this as **AND-SPLIT**); when both A_{1,2} and A_{1,3} terminate, A_{1,4} follows (**AND-JOIN**). On the other hand, when A_{2,1} terminates, only one activity among A_{2,2} and A_{2,3} follows (**OR-SPLIT**); when either A_{2,2} or A_{2,3} terminates, A_{2,4} follows (**OR-JOIN**). When A_{2,4} terminates, A_{2,5} follows without any other splitting or joining activities (**SERIAL**).

An Order Fulfillment Example

Figure 2 provides an order fulfillment process as an example of an inter-organizational process.

Participating organizations include an on-line store and a transportation company, which respectively operate an order handling process model and product delivery process model. PM₁ and PM₂ denote these process models.

In what follows, the overall order fulfillment process is described. When a customer makes an order to purchase a product from the on-line store using her credit card (A_{1,1}), order handling process PM₁ checks the credit card (A_{1,1}). If the credit card is valid, PM₁ notifies the customer that the product will be shipped (A_{1,3}) while it sends a delivery request to the transportation company (A_{1,4}). Note that A_{1,4} in PM₁ is a process activity that stands for remote process model PM₂ and thus A_{1,4} is suspended until PM₂ returns a delivery result.

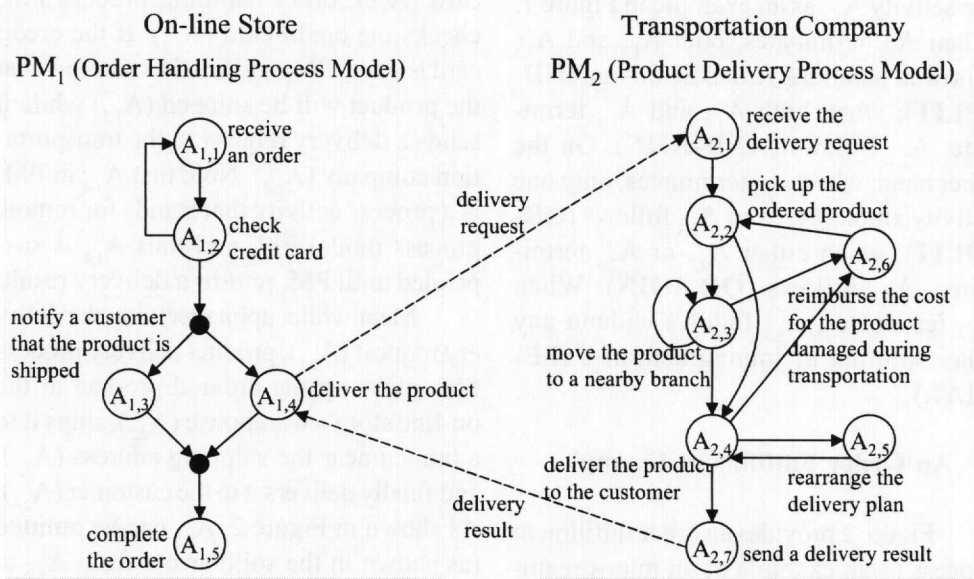
Meanwhile, upon receiving the delivery request (A_{2,1}), product delivery process PM₂ picks up the ordered product at the on-line store's warehouse (A_{2,2}), ships it to a branch near the shipping address (A_{2,3}), and finally delivers it to the customer (A_{2,4}). As shown in Figure 2, A_{2,3} can be omitted (as shown in the solid arrow from A_{2,2} to

$A_{2,4}$) or iteratively executed (the solid arrow from $A_{2,3}$ to $A_{2,3}$) depending on the shipping address and the company's transportation network. If the product cannot be delivered at $A_{2,4}$ because of the customer's absence, PM_2 rearranges the delivery plan to execute $A_{2,4}$ again ($A_{2,5}$). When the product is damaged during transportation at $A_{2,3}$ or $A_{2,4}$, the transportation company reimburses the cost ($A_{2,6}$) and exchanges the damaged product with a new one ($A_{2,2}$). After successfully delivering the product to the customer, a delivery result is sent to PM_1 ($A_{2,7}$) and then PM_1 completes the order ($A_{1,5}$).

If the transportation company provides the on-line store its detailed process status for product delivery, the on-line store can improve the operational efficiency and customer services by accommodating extended order tracking capabilities including both the internal order handling process and external product delivery process. To do so, the on-line store must first integrate the two local process models into one inter-

organizational process model. If the local process models and their interactions are specified explicitly and coordinated by one organization, it is possible to integrate the two local process models (van der Aalst, 1999). However, such an assumption of full information sharing is hardly accepted in the modern business environment because of the autonomy problem stated in the first section. For example, when the transportation company does not want its operational mistakes to be known to customers, it wants to hide the process information related to $A_{2,6}$ (reimbursement for the damaged product). Thus, the transportation company should construct a partial process model by removing $A_{2,6}$ from its product delivery process PM_2 so that the on-line store uses the partial process model instead of full PM_2 to compose an inter-organizational process model. However, such a process information sharing scheme that supports sharing policies among participants has been rarely dealt with in the existing research on inter-organizational processes.

Figure 2: Order Fulfillment Example



The process information sharing system of the on-line store constantly captures and merges the process statuses of the order handling and product delivery processes in order to provide customers the dynamically changing global process status. In doing so, the system refers to the inter-organizational process model including the two process models, PM_1 and PM_2 , and their interactions at the run time, and thus the details of the inter-organizational process model should be encoded in the system. However, achieving agility in companies often causes a change of a process model or sharing policy. When such a change arises, the system should be re-implemented and newly created to reflect the change with maintenance cost and time.

The following sections propose the federated process framework and show how the framework overcomes these problems while facilitating effective implementation of process information sharing.

FEDERATED PROCESS FRAMEWORK

The federated database system approach is referred to in the development of the federated process framework. A federated database system was originally proposed to facilitate information sharing among cooperating but autonomous local database systems (Heimbigner, 1985; Sheth, 1990). While the local database systems independently perform local operations, they only provide partial and controlled data for the requests of the federated database system. Typical development process of the federated database system is composed of the following four steps:

- i) Standardizing local database schemas;
- ii) Controlling and restricting the schemas;

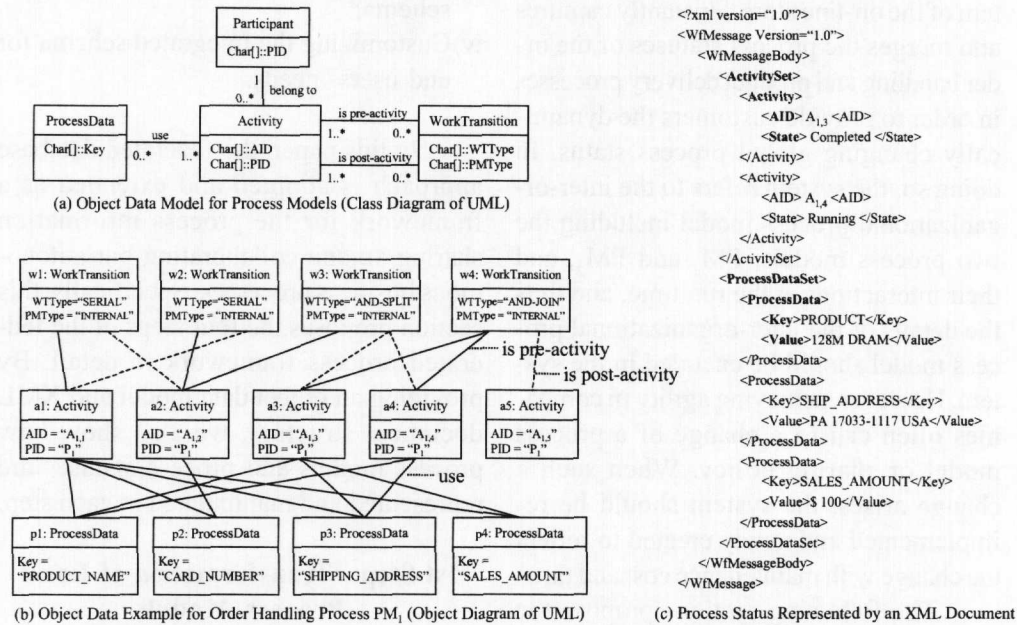
- iii) Integrating the multiple schemas into one schema;
- iv) Customizing the integrated schema for end users' needs.

In this paper, the federated database approach is adopted and extended as a framework for the process information sharing among collaborating but autonomous business processes. Specifically, this section proposes the four steps of the federated process framework in detail. By providing an object data model and XML document structure, we also show how process models and process statuses are represented and manipulated in each step.

1st Step: Transformation of Local Process Models

The first step of the federated process framework is to transform each of the local process models represented by diverse process modeling methods into a semantically equivalent one represented by a canonical standard method. Typically, the process modeling method representing a participant's process model varies depending on the software toolkit or process designer that has been employed to develop the participant's business process system. This diversity of the process modeling methods makes it difficult to integrate different process models (Dabke, 1999; Georgakopoulos, 1999). To address this problem and accommodate seamless integration of different business process models, many standard organizations and researchers have proposed the canonical standard methods for process modeling (Workflow Management Coalition, 1998; Object Management Group, 2000; Bolcer, 1999). As a canonical standard method, this paper uses concepts and notations (e.g., the graphical notations in Figure 1 and 2) of the workflow reference model (Workflow Management

Figure 3: Representation of a Process Model and Process Status Using the UML and XML



Coalition, 1998) proposed by the Workflow Management Coalition because of its popularity among commercial software vendors, and interoperability with other standards.

In Figure 3(a), using the class diagram of the Unified Modeling Language (UML) (Booch, 1999), we define an object data model that acts as a dedicated database schema to manage process models in the OODB. In the UML, a class is represented by a rectangle, containing a class name on the upper side and attributes on the lower side. An association between classes is represented by a line that has an association name and multiplicities on both sides of the connected classes. The *Participant*, *Activity*, and *WorkTransition* classes respectively stand for participants, activities, and work transitions of a process model. The *ProcessData* class stands for process data as a set of data that is referenced or modified by activities of a process model. Note that the *Participant* class has a one-to-many association, “belong to”, with the

Activity class; the *WorkTransition* class has two many-to-many associations, “is pre-activity” and “is post-activity,” with the *Activity* class to respectively indicate its preceding and following activities; the *Activity* class has a many-to-many association, “use,” with the *ProcessData* class to indicate process data referenced or modified by the activity.

Using the object diagram of the UML, Figure 3(b) shows an object data example that represents the order handling process, PM₁, in the form of the presented object data model. An object is represented by a rectangle that has an object name on the upper side and attribute values on the lower side. An association is represented by a line. A solid line and dashed line are used to distinguish “is pre-activity” and “is post-activity” associations between *WorkTransition* and *Activity* objects. For example, *WorkTransition* object w3 represents the AND-SPLIT work transition from A_{1,2} to both A_{1,3} and A_{1,4} in Figure 2.

As an “*is pre-activity*” association, the activity object, *a2*, is linked with a solid line, while objects, *a3* and *a4*, are linked with two dash lines as “*is post-activity*” associations. In this way, the object data example in Figure 3(b) represents the whole structure of the order handling process, PM_1 .

The process status is defined as a set of ongoing activities’ states and a set of process data used by the activities. In PM_1 , suppose that $A_{1,3}$ has notified a customer that an ordered product is being shipped, while $A_{1,4}$ is in the middle of delivering the product. Then, $A_{1,3}$ and $A_{1,4}$ are ongoing activities in the process instance and they use three process data – product, shipping address, and sales amount – according to the object data example in Figure 3(b). Figure 3(c) shows an XML representation of this process status on the basis of the following XML Document Type Declaration (XML DTD) that is a generic XML document structure representing a process status:

```
<! ELEMENT WfMessage (WfMessageBody)>
<! ATTLIST WfMessage Version CDATA #REQUIRED>
<! ELEMENT WfMessageBody (ActivitySet,
ProcessDataSet)>
<! ELEMENT ActivitySet (Activity*) >
<! ELEMENT Activity (AID, State)>
<! ELEMENT AID (#PCDATA)>
<! ELEMENT State (#PCDATA)>
<! ELEMENT ProcessDataSet (ProcessData*) >
<! ELEMENT ProcessData (Key, Value)>
<! ELEMENT Key (#PCDATA)>
<! ELEMENT Value (#PCDATA)>
```

In the beginning, the XML DTD borrows the Wf-XML specification that the Workflow Management Coalition proposed as a standard XML-based representation of a process status (Workflow Management Coalition, 2000), and thus it defines the root element, *WfMessage*, to identify a Wf-XML message and its child element,

WfMessageBody, to indicate a message body section. In representing a process status, however, the Wf-XML covers only the state of a process instance itself such as “running,” “suspended,” “terminated,” etc., and does not support the activity-level process status though an activity is a logical and independent unit of work within a business process. Thus, the proposed XML document structure extends the Wf-XML by taking an activity as a unit of process information sharing. More specifically, an *ActivitySet* element and a *ProcessDataSet* element are defined inside the *WfMessageBody* element. The *ActivitySet* element is to contain all the ongoing activities currently involved (*Activity* element), and every *Activity* element has its identity (*AID* element) and state (*State* element). Similarly, the *ProcessDataSet* element is to contain all the process data used by the ongoing activities (*ProcessData* element), and every *ProcessData* element has its identity (*Key* element) and value (*Value* element). In this way, the tree-structured XML DTD represents activities’ states and associated process data systematically. Moreover, it serves as a generic message specification to facilitate data transfer between different business processes.

In brief, the first step introduces systematic representations of a process model and process status by providing an object data model and XML document structure. Using these representations, the following steps will describe how local process models can be restricted and integrated according to local sharing policies.

2nd Step: Construction of an External Process Model

The second step is to establish a sharing policy and to extract a partial process model from a local process model accord-

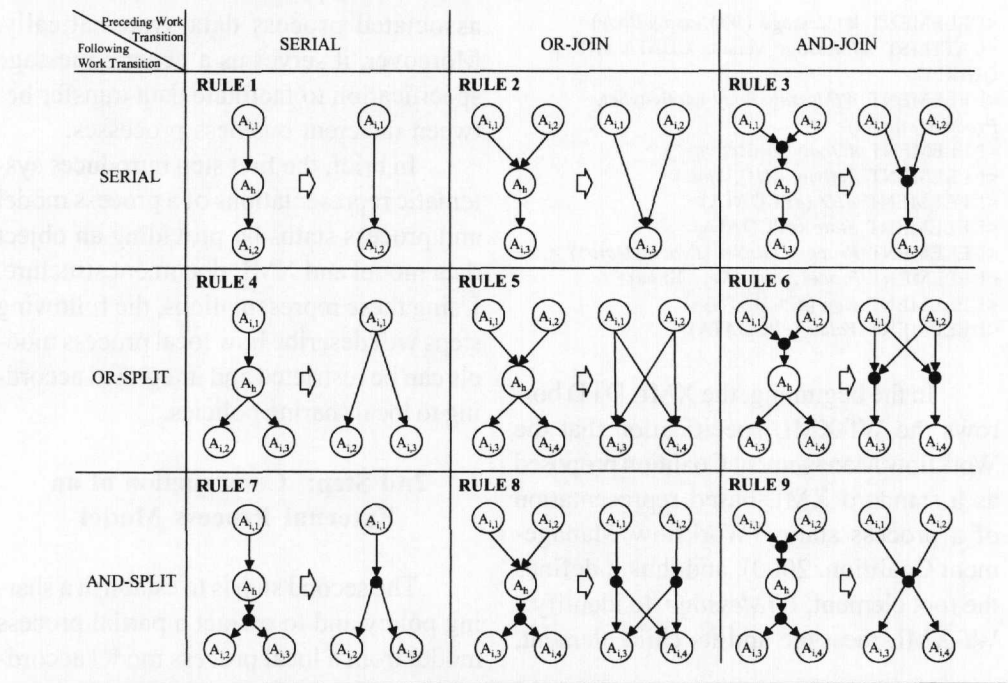
ing to the established sharing policy. Since an activity is a logical unit of process information sharing, the sharing policy is defined as a set of activities shared with other participants. For example, when the transportation company in Figure 2 wants to hide the process information related to $A_{2,5}$ and $A_{2,6}$ from the on-line store, the shared activities are $A_{2,1}$, $A_{2,2}$, $A_{2,3}$, $A_{2,4}$, and $A_{2,7}$. In doing so, the transportation company constructs a partial process model from its original process model, PM_2 , by removing $A_{2,5}$ and $A_{2,6}$, and provides the on-line store with it instead of PM_2 . To distinguish the newly constructed process model from the original one, we call it an **external process model** and the original one an **internal process model**.

Figure 4 presents nine rules to remove a hidden activity for all the possible combinations of a preceding work transition (SERIAL, OR-JOIN, or AND-JOIN) and following one (SERIAL, OR-SPLIT, or AND-SPLIT) of the hidden activities. In

each cell of Figure 4, by removing a hidden activity, A_h , from the left internal process model, the right external process model is generated. The presented rules preserve possible work paths and thus do not distort business logic even though the hidden activity is removed. For example, the left internal process model in RULE 6 has two possible work paths (i.e., after both $A_{i,1}$ and $A_{i,2}$ terminate, $A_{i,3}$ or $A_{i,4}$ follows them through A_h). After A_h is removed, RULE 6 creates two AND-JOINS (i.e., from both $A_{i,1}$ and $A_{i,2}$ to $A_{i,3}$; from both $A_{i,1}$ and $A_{i,2}$ to $A_{i,4}$) in the right external process model and consequently preserves the existing two work paths.

By applying these rules iteratively, an external process model can be constructed from an internal process model. Suppose that, in Figure 2, the transportation company hides $A_{2,5}$ and $A_{2,6}$ from the on-line store. Then, Figure 5(a) shows the procedure that constructs the right external process model from the

Figure 4: Rules for Removing a Hidden Activity



left internal process model, PM_2 , of the transportation company. $A_{2,5}$ is first removed according to RULE 1 and a trigger for self-iteration of $A_{2,4}$ is created. Next, $A_{2,6}$ is removed according to RULE 2 and two triggers, one from $A_{2,3}$ to $A_{2,2}$ and one from $A_{2,4}$ to $A_{2,2}$, are created.

Figure 5(b) presents an object data example that represents both the internal and external process models in Figure 5(a). Note that the *WorkTransition* objects with *PMType* = "INTERNAL" (*w1* to *w11*) denote the work transitions in the internal process model, and the *WorkTransition* objects with *PMType* = "EXTERNAL" (*w12* to *w20*) denote the work transitions in the external process model. Using the *WorkTransition.PMType* attribute, we can extract shared activities from the object data example, and thus this object data example shows how to represent the transportation company's sharing policy toward

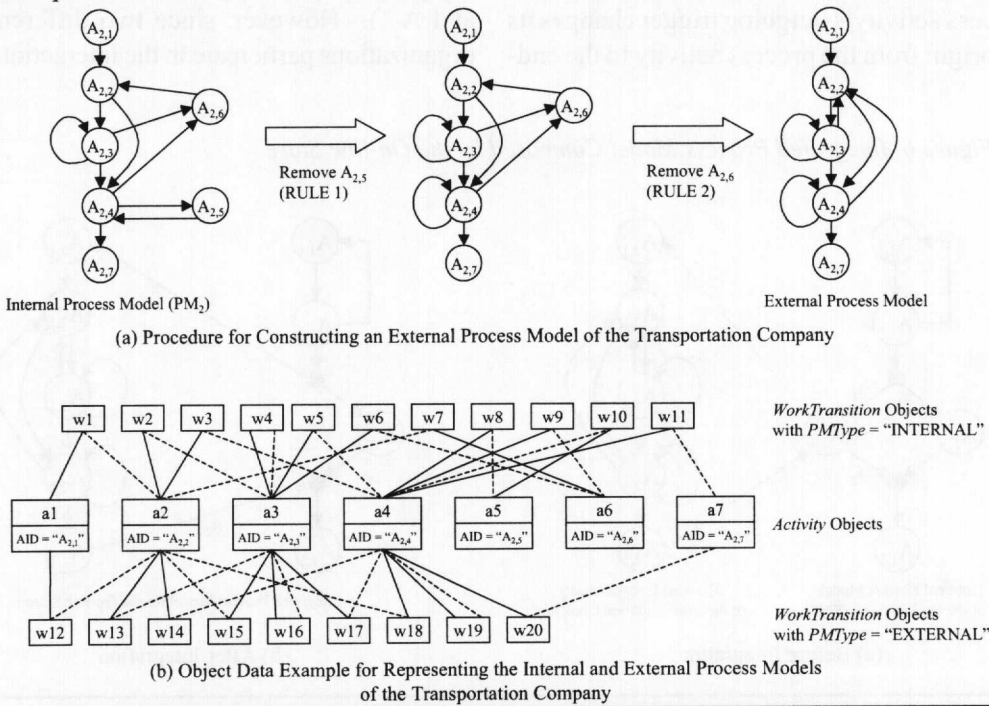
the on-line store. In terms of the XML document, an XML document based on the internal process model can be easily transformed into an XML document based on the external process model by removing *Activity* elements corresponding to hidden activities.

To accommodate the activity-level sharing policy, this step introduces the concept of an external process model and its construction method, and furthermore describes how to represent the sharing policy and corresponding external process model in the presented object data model. As a result, this step provides an answer to the first question of the autonomy problem.

3rd Step: Composition of an Integrated Process Model

This step describes a method for a participant to merge its internal process

Figure 5: Construction of an External Process Model and Corresponding Object Data Example



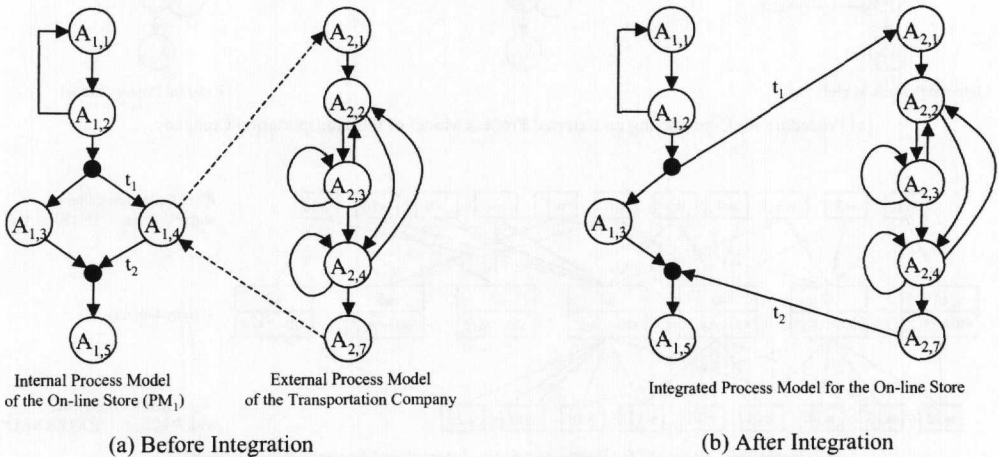
model and an external process model provided by another participant. Such a merged process model is called an **integrated process model**. The method to compose the integrated process model is to substitute the process activity in the internal process model with the corresponding external process model. An earlier section stated that the internal process model uses a process activity to represent the activities of the external process model. Figure 6(a) shows that the process activity, $A_{1,4}$, of PM_1 represents the activities of the external process model of the transportation company. $A_{1,4}$ sends a request message to the starting activity, $A_{2,1}$, of the external process model, and receives a result message from the ending activity, $A_{2,7}$, of the external process model.

The substitution of the process activity with the corresponding external process model is performed in the following ways: first, the process activity's incoming trigger changes its destination from the process activity to the starting activity of the external process model. Second, the process activity's outgoing trigger changes its origin from the process activity to the end-

ing activity of the external process model. Finally, the process activity is removed from the internal process model. For example, t_1 and t_2 in Figure 6(a) are $A_{1,4}$'s incoming and outgoing triggers, respectively. To constitute the integrated process model from the two process models in Figure 6(a), we change t_1 's destination from $A_{1,4}$ to $A_{2,1}$ and t_2 's origin from $A_{1,4}$ to $A_{2,7}$, and finally remove $A_{1,4}$ from PM_1 . Figure 6(b) shows the resultant integrated process model.

Using the *PMType* attributes of the *WorkTransition* objects, the object data example in Figure 5(b) showed how the external process model could be represented with the object data model in Figure 3(a). Similarly, the integrated process model can be represented with the object data model by assigning "INTEGRATED" to the *PMType* attributes of the *WorkTransition* objects related to the integrated process model. Then, an interaction between different process models (e.g., the work transition between $A_{1,4}$ and $A_{2,1}$) is represented in the same way as a work transition within a process model (e.g., one between $A_{2,1}$ and $A_{2,2}$). However, since two different organizations participate in the interaction,

Figure 6. Integrated Process Model Composed by the On-line Store



the interaction has special features such as a distributed transaction and a separated role and responsibility model. The ebXML Business Process Specification Schema (BPSS) (UN/CEFACT, 2001) that provides a detailed UML class diagram to design an interaction between collaborating business entities supports a distributed transaction and a separated role and responsibility model by considering an interaction as a business transaction between activities. Thus, the object data model in Figure 3(a) can be extended by inheriting a specialized interaction class from the *WorkTransition* class and connecting the interaction class and the UML class diagram of the ebXML BPSS, and consequently, the extended data model would describe the interactions more precisely in terms of a distributed transaction and a separated role and responsibility model. However, to simplify the federated process framework and focus on the research questions, this paper will use the object data model in Figure 3(a) in the remaining parts without extending it.

Meanwhile, the XML document based on an integrated process model can be derived by merging XML documents based on internal or external process models comprising the integrated process model. To merge XML documents based on different process models, each of their *ActivitySet* elements and *ProcessDataSet* elements is merged separately. Particularly, an *Activity* element corresponding to a process activity should be deleted since the process activity does not exist in the integrated process model. Figure 7 shows an example of merging the left XML document representing the on-line store's order handling process status and the right XML document representing the transportation company's product delivery process status. The resultant XML document in the middle of Figure 7 represents the global

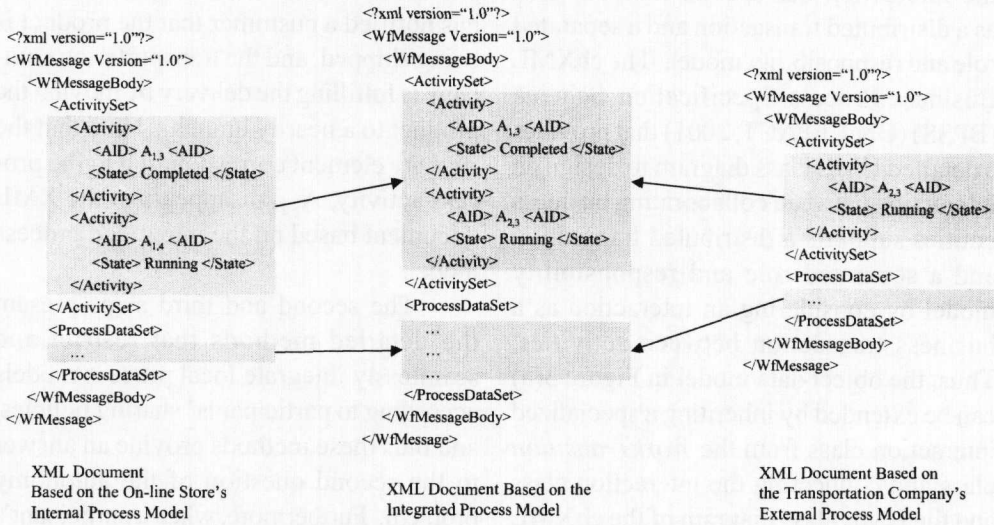
process status that means "the on-line store has notified a customer that the product is being shipped, and the transportation company is fulfilling the delivery by moving the product to a nearby branch." Note that the *Activity* element corresponding to the process activity, $A_{1,4}$, disappears in the XML document based on the integrated process model.

The second and third steps present the detailed methods that restrict and seamlessly integrate local process models according to participants' sharing policies, and thus these methods provide an answer to the second question of the autonomy problem. Furthermore, when a participant's process model or sharing policy is changed, these methods automatically generate external and integrated process models. Thus, the second and third steps also provide base algorithms to solve the agility problem.

4th Step: Construction of a Customized Process Model

The final step of the federated process framework is to construct a partial process model from an integrated process model. The integrated process model can be quite large and complex, and thus it may need to be reduced into a partial process model to support access control for user-level securities or to provide a smaller process model appropriate to a specific task of a user or user group. Such a partial process model constructed from an integrated process model is called a **customized process model**. When a customized process model is constructed, users' needs for customization are first identified, and activities that are unnecessary for the needs are removed. To remove the unnecessary activities, the nine rules presented in the second step are iteratively applied to the integrated process model. Figure 8 exem-

Figure 7: XML Document Based on the Integrated Process Model in Figure 6(b).



plifies a customized process model for the on-line store's customer support staff, which is constructed by removing messaging activities, $A_{2,1}$ and $A_{2,7}$ from the integrated process model in Figure 6(b). This customized process model helps the customer support staff to focus only on the process information necessary for customer services.

The existing object data example can be easily extended in order to manage the customized process model by adding new *WorkTransition* objects for the customized process model and assigning "CUSTOMIZED" to their *PMType* attributes. On the other hand, in the same way as in the second step, an XML document based on the integrated process model can be transformed into one based on the customized process model by eliminating the *Activity* elements corresponding to the removed activities.

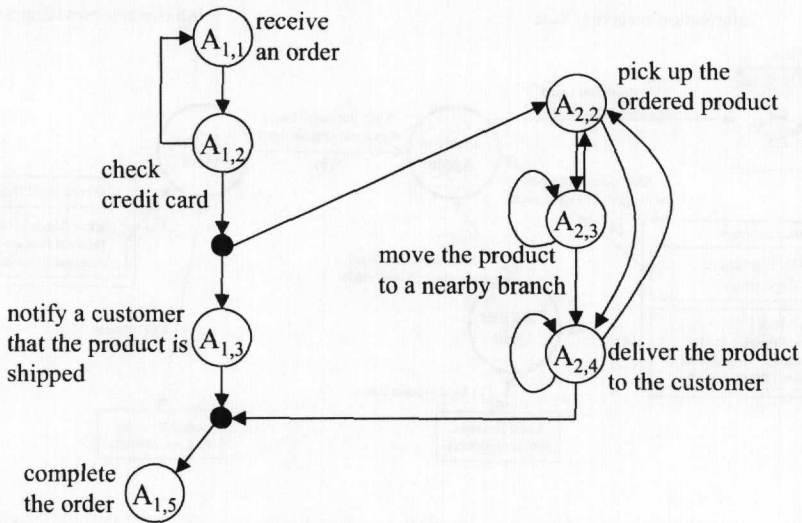
This section presents the federated process framework to facilitate process information sharing among collaborating but autonomous organizations. Specifically, the

four steps of the framework facilitate seamless integration of multiple collaborating process models while accommodating flexible activity-level sharing policies. To support these features of the framework systematically, this section also presents the object data model to manage a process model and the XML document structure to represent process status. Therefore, the proposed federated process framework provides answers to the two questions of the autonomy problem and consequently addresses the autonomy problem.

SYSTEM ARCHITECTURE AND ITS PROTOTYPE SYSTEM

This section presents a system architecture based on the federated process framework with its prototype system. The system architecture is composed of the following two types of system components: a repository and an agent. Figure 9 shows the components and their interactions. In terms of the repository, the system architecture contains process model repository

Figure 8: Customized Process Model for the On-line Store's Customer Support Staff.



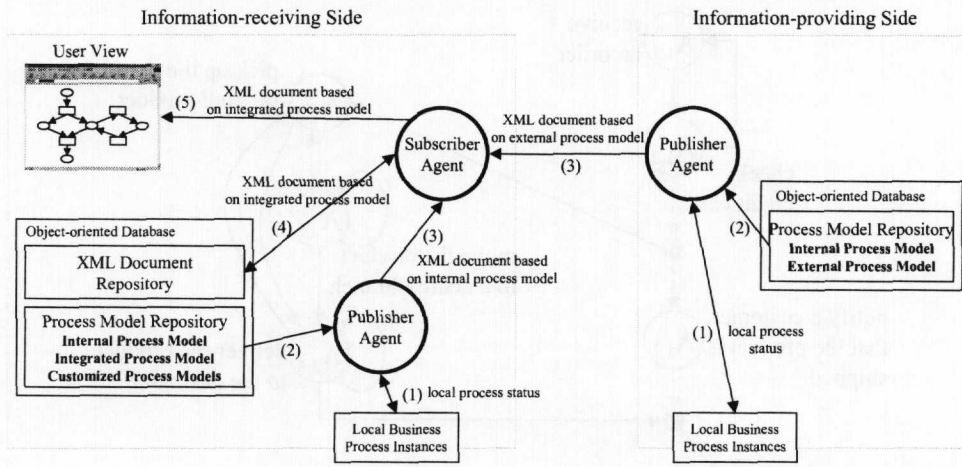
ries and an XML document repository. Employing the object data model in Figure 3(a), the **process model repository** manages internal, integrated, and customized process models on the information-receiving side, and it manages internal and external process models on the information-providing side. The **XML document repository** resides only on the information-receiving side and manages a set of XML documents that represent the execution history of global process instances.

In terms of the agent, the system architecture contains publisher agents and a subscriber agent. On both sides, the **publisher agent** detects the change of a local process status and sends the XML document representing the changed local process status to the subscriber agent that resides on the information-receiving side. The **subscriber agent** manages the XML document repository and broadcasts it to relevant user views. When the subscriber agent receives an XML document from a publisher agent, it first refers to the latest XML document based on the integrated process model from the XML document

repository, modifies it according to the received XML document, and inserts the modified XML document based on the integrated process model into the XML document repository. Then, the subscriber agent sends the modified XML document to the relevant user views that visually provide end users current global process status.

The numbered arrows in Figure 9 show the change notification procedure ((1) to (5)) in detail. In the proposed system architecture, the change notification procedure is encoded in the application programs such as the agents and user view, but participants' process models and sharing policies are managed in the process model repository. Thus, the system procedures in the application programs are independent of contents of the process model repository. When a participant's process model or sharing policy is changed, the associated external and integrated process model are automatically generated through the second and third steps of the federated process framework. Then, the proposed system architecture can adapt itself to the change just by revising the contents of the

Figure 9 System Architecture Based on the Federated Process Framework



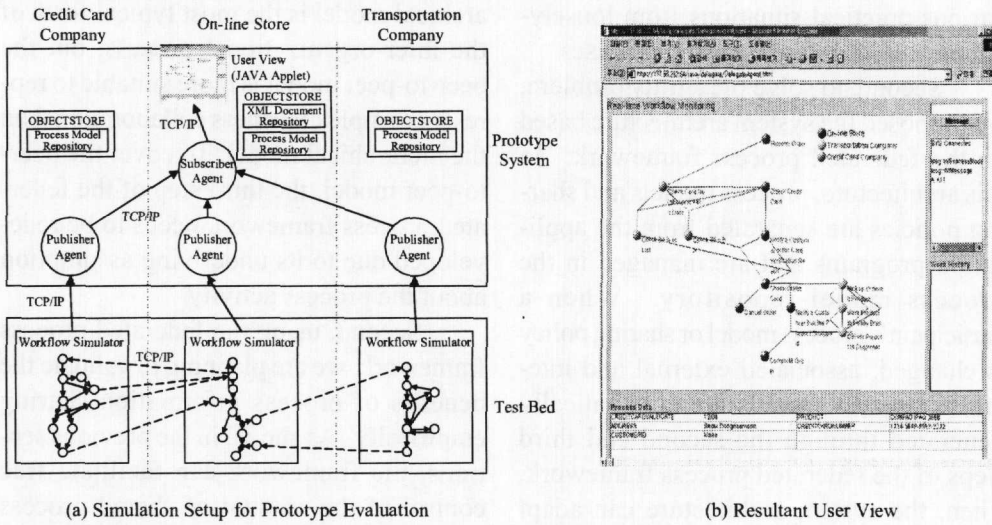
process model repository. This adaptability of the system architecture provides the answer to the first question of the agility problem; in addition, the system components comprising the system architecture provides the answer to the second question of the agility problem. Consequently, the system architecture based on the federated process framework addresses the agility problem.

Using the system architecture, a prototype system was also developed based on a commercial OODB called OBJECTSTORE with the JAVA programming language. In the prototype system, we employed a real inter-organizational process model that is composed of three collaborating internal process models including the on-line store's order handling process, transportation company's delivery process, and credit card company's card verification process. The internal process models are managed separately in different process model repositories that were built up in the OBJECTSTORE as the form of the object diagram in Figure 3(b). The agents and a user view, which were implemented using the JAVA programming lan-

guage, access the process model repositories and interact with each other through the TCP/IP network. The lower part of Figure 10(a) shows a test bed that imitates the executions of the three business processes to evaluate the prototype system. The adopted test bed contains three workflow simulators corresponding to the three participating internal process models. According to the internal process models and their interactions, these workflow simulators proceed simultaneously as independent software processes while interacting with each other. Whenever a workflow simulator's state is changed, the workflow simulator sends an event to a publisher agent and then the change notification procedure is performed. Figure 10(b) shows the JAVA applet-based user view that provides real-time process status on the on-line store's customized process model.

All the system components of the prototype system were tested for accuracy and completeness by changing the sharing policy for the amount of shared process information (i.e., changing the external and integrated process models in the process model repositories). Furthermore, by per-

Figure 10: Evaluation of the Prototype System



forming scenario experiments for the change of an internal process model or a sharing policy, the prototype system was evaluated in terms of the adaptability. Consequently, the prototype system shows that it is both possible and feasible to develop the federated process framework while solving the autonomy and agility problems.

CONCLUSIONS AND FUTURE RESEARCH

The main contribution of this paper is that, as a conceptual design to facilitate process information sharing and to address the autonomy and agility problems, we provide the federated process framework and its system architecture that are adequate for the needs of the modern business environment and current technologies. Recall that the initial goal of the paper was to provide a system framework to facilitate process information sharing in the modern business environment. Specifically, at the beginning of the paper, we issued the autonomy and agility problems. These two problems have not been previously ad-

ressed in the literature though they are crucial obstacles to developing process information sharing. To achieve an efficient and applicable framework, this paper makes efforts to address these problems in the following ways.

First, to solve the autonomy problem, we proposed the federated process framework. Specifically, the first step of the framework provides an object data model and XML document structure as systematic representations of a process model and process status. The second step distinguishes the external process model from the internal process model and provides the detailed rules to automatically generate the external process model. The third step provides the method to seamlessly integrate multiple process models according to participants' sharing policies. The fourth step tailors the integrated process model for the purposes of customization and user-level access control. In this way, the four steps facilitate the activity-level sharing policy and seamless process model integration. Consequently, the federated process framework overcomes the autonomy prob-

lem by supporting flexible sharing policies, and accommodates wide applicability to various practical situations from loosely-coupled cases to tightly-coupled cases.

Second, to solve the agility problem, we proposed the system architecture based on the federated process framework. In this architecture, process models and sharing policies are separated from the application programs and are managed in the process model repository. When a participant's process model or sharing policy is changed, associated external and integrated process models are automatically generated through the second and third steps of the federated process framework. Then, the system architecture can adapt itself to the change just by revising the contents of the process model repository. The presented object data model and XML document structure make it possible to achieve such adaptability by providing conceptual designs that can be extended for the actual development of the process model repository and change notification procedure. As a result, the system architecture addresses the agility problem and allows sufficient adaptability to support the entire lifecycle of process information sharing. By adding physical system capabilities such as security control and data conflict resolution, the proposed system architecture can be extended as a fully fledged physical system design for process information sharing.

In future research, first, we are extending the federated process framework to cover the peer-to-peer model (Workflow Management Coalition, 1998) of the inter-organizational process in which local processes exchange asynchronous messages at their runtime. In presenting the federated process framework, this paper focuses on the hierarchical model (Workflow Management Coalition, 1998) in which a local

process uses a process activity to represent other collaborating process. The hierarchical model is the most typical form of the inter-organizational process, but the peer-to-peer model is more suitable to represent complex business collaboration than the hierarchical model. To cover the peer-to-peer model, the third step of the federated process framework needs to be redeveloped due to its underlying assumption about the process activity.

Second, using the federated process framework, we are planning to validate the benefits of process information sharing empirically. As shown in the previous sections, the framework can facilitate free control of the amount of shared process information. Thus, it provides a test bed to simulate various situations of process information sharing with respect to the amount of shared process information. For example, in the information-providing side, a process instance is simulated according to real stochastic distributions for state transitions among activities, and its process information is delivered to the information-receiving side. On the information-receiving side, the dynamic scheduling method uses the delivered process information to optimize the execution of an internal process instance. By controlling the amount of shared process information in the information-providing side, we expect to investigate the relationships between the amount of shared process information and the performance of the dynamic scheduling method empirically.

REFERENCES

- Alonso, G. et al. (1999). WISE: Business to Business E-Commerce. In *Proceedings of the 9th IEEE International Workshop on Research Issues on Data Engineering. Information Technology*

- for *Virtual Enterprises (RIDE-VE 1999)*, Sydney, Australia, 23-24.
- Alonso, R. & Barbara, D. (1989). Negotiating Data Access in Federated Database Systems. In *Proceedings of the 5th International Conference on Data Engineering (February)*, California, USA, 56-65.
- Amghar, Y. et al. (2000). Using Business Rules Within a Design Process of Active Databases. *Journal of Database Management*, 11(3), 3-15.
- Arnold, K. et al. (2000). *The Java Programming Language*. California: Addison-Wesley.
- Bakos, Y. (1998). The Emerging Role of Electronic Marketplaces on the Internet. *Communications of ACM*, 41(8), 35-42.
- Bolcer, G.A. & Kaiser, G. (1999). SWAP: Leveraging the Web to Manage Workflow. *IEEE Internet Computing*, 3(1), 85-88.
- Böhm, K. (2000). On extending the XML engine with query-processing capabilities. In *Proceedings of Advances in Digital Libraries (ADL)*, Bethesda, Maryland, 127-138.
- Booch, G. et al. (1999). *The Unified Modeling Language User Guide*. Boston: Addison-Wesley.
- Carter, B. (2000). XML: Filling a Data-Description Gap, Part II. *Journal of Database Management*, 11(2), 30-33.
- Casati, F. et al. (1998). Workflow Evolution. *Data & Knowledge Engineering*, 24(3), 211-238.
- Chircu, A.M. & Kauffman, R.J. (2000). Reintermediation Strategies in Business-to-Business Electronic Commerce. *International Journal of Electronic Commerce*, 4(4), 7-42.
- Cichocki, A. et al. (1998). *Workflow and Process: Concepts and Technology*. Boston: Kluwer Academic Publishers.
- D'Amours, S. et al. (1999). Networked Manufacturing: The Impact of Information Sharing. *International Journal of Production Economics*, 58, 63-79.
- Dabke, P. (1999). Enterprise Integration via CORBA-based Information Agents. *IEEE Internet Computing*, 3(5), 49-57.
- Georgakopoulos, D. et al. (1999). Managing Process and Service Fusion in Virtual Enterprises. *Information Systems*, 24(6), 429-456.
- Goranson, H.T. (1999). *The Agile Virtual Enterprise: Cases, Metrics, Tools*. London: Quorum Books.
- Heimbigner, D. & McLeod, D. (1985). A Federated Architecture for Information Management. *ACM Transactions of Office Information Systems*, 3(3), 253-278.
- Herring, C. & Milosevic, Z. (2001). Implementing B2B Contracts Using BizTalk. In *Proceedings of the 34th Hawaii International Conference on System Sciences*, Hawaii, USA, 4078-4087.
- Kim, W. (1995). *Modern Database Systems: The Object Model, Interoperability, and Beyond*. New York: Addison-Wesley.
- Kuechler, W. et al. (2001). Supporting Optimization of Business-to-Business E-Commerce Relationships. *Decision Support Systems*, 31, 363-377.
- Lee, H.B. et al. (1999). KWM: Knowledge-based Workflow Model for Agile Organization. *Journal of Intelligent Information Systems*, 13(3), 261-278.
- Lee, H.L. et al. (1997). The Bullwhip Effect in Supply Chains. *Sloan Management Review*, 38, 93-102.
- Leymann, F. & Roller, D. (2000). *Production Workflow: Concepts and Techniques*. Englewood Cliffs NJ: Prentice Hall.
- Merz, M. & Lamersdorf, W. (1999). Crossing Organizational Boundaries with

Mobile Agents in Electronic Service Markets. *International Journal on Computer-Aided Engineering*, 6(2), 91-104.

Mori, M. et al. (1999). Proposal of Application Architecture in Electronic Commerce Service between Companies. In *Proceedings of the International Workshop on Advance Issues of E-Commerce and Web-Based Information Systems (WECWIS)*, Santa Clara, USA, 46-49.

Object Design. (2001). *ObjectStore*. <http://www.odi.com/>.

Object Management Group. (2000, April). *Workflow Management Facility Specification V1.2*. [Online]. Available: <http://www.omg.org/>.

Scott-Morton, M.S. (1994). The 1990s Research Program: Implications for Management and the Emerging Organization. *Decision Support Systems*, 12(2), 251-256.

Sheth, A.P. & Larson, J.A. (1990). Federated Database Systems for Managing Distributed, Heterogeneous, and Autonomous Databases. *ACM Computing Surveys*, 22(3), 183-236.

Sundaram, M. & Shim, S.S.Y. (2001). Infrastructure for B2B exchanges with RosettaNet. In *Proceedings of the International Workshop on Advance Issues of E-Commerce and Web-Based Infor-*

mation Systems (WECWIS), California, USA, 110-119.

Tuma, A. (1998). Configuration and Coordination of Virtual Production Networks. *International Journal of Production Economics*, 56, 641-648.

UN/CEFACT & OASIS. (2001, May 11). *Business Process Specification Schema V1.01*. [Online]. Available: <http://www.ebxml.org/>.

van der Aalst, W.M.P. (1999). Process-oriented Architectures for Electronic Commerce and Interorganizational Workflow. *Information Systems*, 24(8), 639-671.

Workflow Management Coalition. (1998, August 5). *Interface 1: Process Definition Interchange Process Model*, Document Number WfMC-TC-1016-P. [Online]. Available: <http://www.wfmc.org/>.

Workflow Management Coalition. (2000, May 1). *Workflow Standard – Interoperability Wf-XML Binding*, Document Number WfMC-TC-1023. [Online]. Available: <http://www.wfmc.org/>.

Zhou, Q. et al. (1998). An Information Management System for Production Planning in Virtual Enterprises. *Computers & Industrial Engineering*, 35(1-2), 153-156.

Kyoungil Bae is a consultant at the PricewaterhouseCoopers Consulting. He received a Ph.D. in management information systems at the Korea Advanced Institute of Science and Technology (KAIST), Seoul. His research includes synchronous collaboration in groupware/virtual enterprises, system integration in the heterogeneous computing environments, workflow and process automation, and customer relationship management. His e-mail address is kibae@kgsm.kaist.ac.kr.

Junghyun Kim is a research assistant at the Applied Database Lab at the KAIST. She received an M.S. in Information Systems from the University of Colorado at Denver, Colorado, USA. Her research interests are in process model and data mining. Her e-mail address is jh0kim@kgsm.kaist.ac.kr.

Soonyoung Huh is an assistant professor of Management Information Systems at

the KAIST. He received a Ph.D. from the University of California, Los Angeles. Dr. Huh has published articles in such journals as Decision Sciences, International Journal of Intelligent Systems in Accounting, Finance and Management, Omega, Journal of Systems and Software. His research deals with abstraction database application to business systems, model management, object-oriented database approach to decision support systems, and intelligent approaches in financial trading systems. His e-mail address is syhuh@kgsm.kaist.ac.kr.